

BTG Admin Guide: Projects Portal

Audience: BTG administrator who collect architecture diagrams and IdaC firewall workbooks from delivery teams, link them to portal records, align JIRA firewall work with projects, run LLM-assisted analysis, and keep ARB tickets consistent with linked artefacts.

Environment: The Node analysis service runs on an internal host that can reach your JIRA and LLM endpoints. The portal calls that service over HTTPS, including where access is provided through a tunnel. Production should use valid JIRA and LLM configuration on that host.

1. Capabilities

Capability	Location
Create, edit, or archive projects	Projects Portal grid
Search JIRA projects to set Project Code	New or Edit Project modal
Search and attach an ARB JIRA ticket	New or Edit Project modal
Assign team and people	Edit Project modal
Link a .drawio diagram	Card action Link Diagram
Link an IdaC .xlsx workbook exported from the canvas	Card action Link IdaC workbook
Refresh the linked ARB description in JIRA after links change	Automatic after each successful link, with toast feedback
Open a linked diagram in the editor	Card action Open in Canvas
Download linked IdaC workbook	Card strip on the workbook row
Start JIRA polling and LLM analysis	Header Run Analysis
Send project metadata to the service	Header Export Snapshot
Check service status	Header status indicator
Read LLM outcomes per project	Card Analysis panel

2. Opening the portal

You may use any of these entry points. The user interface is the same.

- **Web application:** the HTTPS URL where `projects.html` is served (for example `https://portal.example.org/projects.html`).
- **Hosted:** your internal draw.io deployment URL supplied by IT.

Connecting the portal to the analysis service

Assume the **service** Node application runs on a real host with an HTTPS base URL, for example <https://analysis.internal.example.com>, and that the **static web deployment** exposes the portal and **LLM Analysis** with scripts configured to use that same API base URL for every call.

1. Run **service** where it can reach JIRA and the LLM API.
2. Publish it on TLS with a hostname browsers can resolve per your security model.
3. Configure the front-end build or publish step so the injected **service base URL** matches that hostname. Then **Export Snapshot**, **Run Analysis**, JIRA helpers, ARB updates, status checks, and chat all target one API origin.
4. On the API host, allow the **Origin** of the static web UI in CORS for the **GET** and **POST** routes under **/api** that the portal uses.

For local development only, the same scripts may point at <http://localhost:3001> or another loopback URL.

Project metadata and linked binaries stay in the browser under **localStorage** key **hkma.projects**. Run **Export Snapshot** after material changes so the service keeps correct correlation between JIRA items, projects, and ARB references.

3. Create or edit a project

1. Choose **+ New Project** or **Edit** on a card.
2. Complete the modal:
 - **Project Name:** required.
 - **Search JIRA projects:** pick a row so **Project Code** fills with the JIRA project key. If the service is offline, type the code manually.
 - **Search ARB tickets:** optional Architecture Review key so the card can show ARB metadata and drive ARB updates on link.
 - **Owning Team:** AS1, AS2, ITIS, ITDP, ITPSO, BTG, BSA, ITS, or Other.
 - **Status:** Active, On Hold, or Archived.
 - **People Assigned:** comma-separated names.
 - **Description:** free text.
3. Choose **Save Project**.

Cards sort by status with Active first, then On Hold, then Archived, then by newest update within each group.

4. Link diagrams and IdaC workbook

4.1 Link Diagram

1. Obtain the **.drawio** file from the project team. See the [Project Team Guide](#).
2. Choose **Link Diagram** on the card.
3. Select the file in the system picker.
4. The card shows a strip with **Open in Canvas**, **Download .drawio**, and **unlink**. **Open in Canvas** stores a short-lived draft in **localStorage** and opens **index.html** with a draft key.

If an ARB ticket is on the project, the service calls **POST /api/jira/arb-update** with filenames and timing. Success appears in a toast; failures are non-blocking and explain the error.

4.2 Link IdaC workbook

1. Obtain the IdaC `.xlsx` from **Export firewall** with the IdaC template on the canvas **File** menu.
2. Choose **Link IdaC workbook**.
3. Select the `.xlsx`. Invalid files trigger an error toast.
4. The card shows a second strip with download and unlink for the workbook.

Re-run **Export Snapshot** after meaningful edits so correlation and LLM context stay accurate.

5. Search, filter, edit, delete

- **Edit**: opens the modal with current values including JIRA and ARB fields when configured.
- **Delete**: asks for confirmation, then removes the project and its stored attachments from `localStorage`.
- **Search**: matches name, code, team, members, and description.
- **Status** and **Team** filters narrow the grid.

The toolbar shows how many projects match when filters apply.

6. JIRA polling and LLM analysis in production

The analysis service:

1. **Polls JIRA** on the configured cadence for open firewall request issues. JQL is configurable and may span projects.
2. **Correlates** each issue to a portal project using project key, name similarity, and optional hints.
3. **Builds LLM context** including effective architecture rules from the service data path.
4. **Calls the LLM** with retries on transient errors. Failures surface as `llmError` so you do not mistake an outage for approval.
5. **Stores** results for `GET /api/analysis/:projectId`.

6.1 Service setup on the internal host

```
cd service
copy .env.example .env
npm install
npm start
```

Set JIRA URL, credentials, LLM URL, LLM API key, and optional `JIRA_FIREWALL_JQL` in `.env`. `SERVICE_PORT` is the listen port inside the process and defaults to 3001. Production usually fronts this with a reverse proxy and HTTPS; the port is whatever your image or unit exposes behind that proxy.

Use real JIRA and LLM endpoints in production. Blank JIRA or LLM URLs enable demonstration behaviour with fixture data only on isolated demo machines.

6.2 Export Snapshot

1. Choose **Export Snapshot**.

2. The browser downloads `projects-snapshot.json` and **POSTs** the trimmed payload to `/api/projects/sync`.
3. A toast confirms delivery. If the service is unreachable, keep the downloaded file for manual placement under `service/data` on the analysis host when your process allows it.

Export again when codes, ARB keys, or membership change in ways that matter for routing work.

6.3 Run Analysis

1. Choose **Run Analysis**.
2. The portal **POSTs** `/api/run`. If a run is already in progress, the API may return `skipped` and the UI should say so.
3. When polling finishes, cards refresh from `GET /api/analysis/:id`. The header status shows last and next scheduled run.

6.4 Reading card outcomes

Indicator	Meaning
Approved	LLM and schema context support allowing the request.
Clarification	Ambiguity or policy gap needs reviewer input.
Pending	Default path pending human review.
LLM error	The model path failed after retries; check logs and retry.

Demonstration stacks may annotate analysis timestamps when results are illustrative.

7. Common workflows

Workflow A: New package from an architect

1. Receive the `.drawio` and matching IdaC `.xlsx`.
2. Create or open the project. Use JIRA search to bind the project key and optional ARB ticket.
3. **Link Diagram** and **Link IdaC workbook**.
4. Confirm ARB update toast if applicable.
5. **Open in Canvas** for a quick review if needed.
6. **Export Snapshot**, then **Run Analysis**, then read each **Analysis** strip.

Workflow B: Daily triage

1. Open the portal. Confirm status shows the service online.
2. Filter **Active** projects.
3. For each **Clarification** item, open the JIRA issue, inspect the workbook row, and add comments as your process requires.

Workflow C: Onboard a new BTG administrator

1. Share this guide and the [Screenshots Walkthrough](#).
2. Practice on a laptop demo stack if your organisation provides one.

3. Move to the internal analysis host with read-only JIRA and a single test issue before enabling comment posting to JIRA.

8. Troubleshooting

Symptom	Likely cause	What to try
Status offline	Service stopped, wrong URL, or network	Start service , verify URL, VPN, and CORS
JIRA search empty	Auth failure or API block	Verify token and enterprise firewall rules
Run Analysis appears idle	Duplicate trigger while a run is active	Read toast and status for skipped
LLM error text	Timeout or quota after retries	Check LLM logs and capacity
ARB toast shows failure	Invalid key or missing write permission	Verify ticket id and service account scope
Canvas opens blank	Draft key already consumed	Choose Open in Canvas again
IdaC rejected	Wrong type or corrupted file	Re-export from the canvas
No cards after save	Browser blocked storage	Adjust privacy settings

9. Where data is stored

Data	Storage
Project records	Browser localStorage key hkma.projects
Linked diagram XML	Inside each project record
Linked IdaC workbook	Inside each project record as Base64 plus filename and timestamps
ARB and JIRA linkage fields	On each project record per your shipped portal build
Open-in-canvas drafts	localStorage keys prefixed hkma.opendraft
Snapshot on the server file system	service/data/projects-snapshot.json
Analysis results file	service/data/analysis-results.json
Per-diagram schema overrides	localStorage and attributes inside the .drawio file

Field names on each project record match the portal build your organisation deploys.

10. Related material

- [Screenshots Walkthrough](#) for captioned figures across portal, LLM chat, canvas, and firewall panel.
- [Project Team Guide](#) for diagram and IdaC production.
- [Reviewer Guide](#) for LLM assisted conversational review.